

Redes Neurais Convolucionais (CNN)

Guillermo Cámara Chávez

Por que os Dados são Importantes?

- Discussão sobre a importância da qualidade e da organização dos dados no treinamento de modelos de *machine learning*.
- **Desafio:** Dados desorganizados ou mal processados podem levar a um desempenho ruim, mesmo com um modelo bem projetado.
- Breve introdução ao pipeline de treinamento de um modelo: *Dataset -> DataLoader -> Modelo -> Treinamento*.

O que são Datasets no PyTorch?

- Introdução à classe base *torch.utils.data.Dataset*.
- *Datasets* são usados para representar e manipular grandes conjuntos de dados.
- Principais responsabilidades:
 - Organizar os dados em uma estrutura acessível.
 - Fornecer dados de entrada ao modelo durante o treinamento.

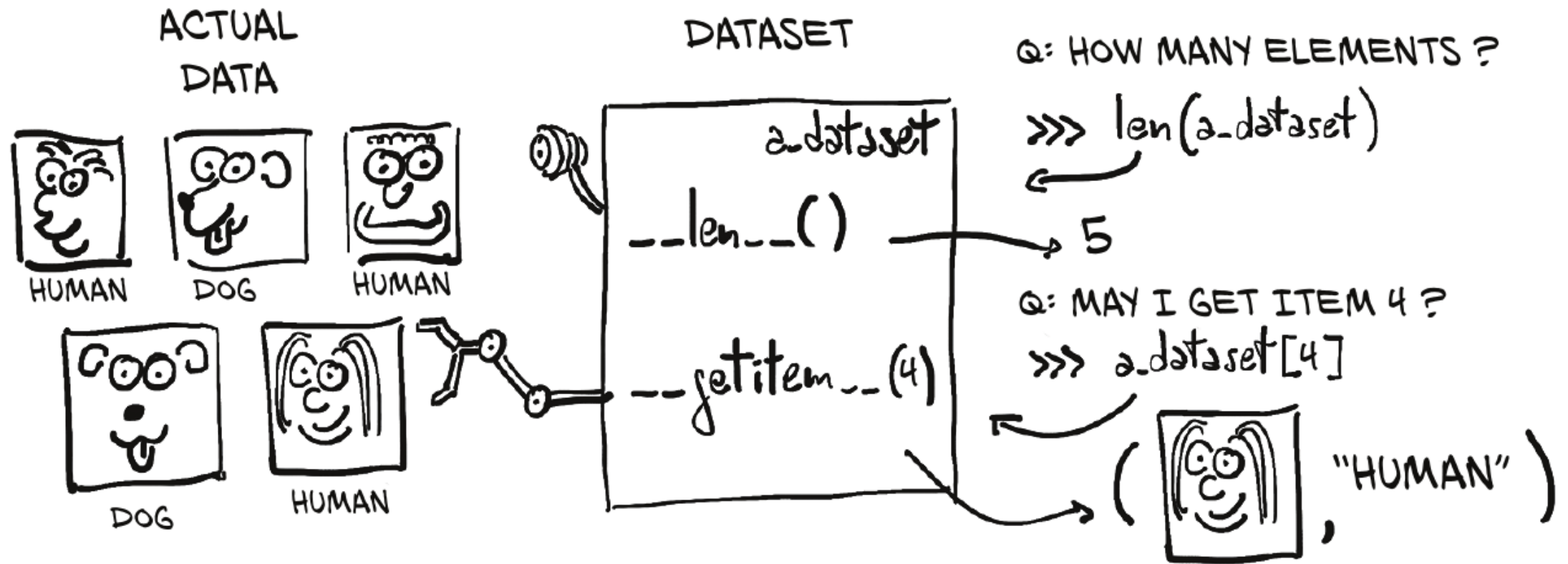
O que são Datasets no PyTorch?

- Datasets são abstrações para carregar e manipular dados.
- PyTorch fornece a classe base `torch.utils.data.Dataset`.
- Dois tipos de Datasets:
 - Pré-construídos (e.g., MNIST, CIFAR10)
 - Personalizados (quando temos dados próprios)

Estrutura da Classe Dataset Personalizado

- Quando os datasets prontos não são suficientes, precisamos criar nossos próprios datasets.
- A classe *Dataset* precisa implementar três métodos principais:
 - `__init__`: Inicializa o dataset e carrega os dados.
 - `__len__`: Retorna o tamanho do dataset.
 - `__getitem__(idx)`: Acessa um exemplo específico através do índice `idx`.

Estrutura da Classe Dataset Personalizado



Datasets e Dataloaders

```
from torch.utils.data import Dataset
```

```
class MeuDataset(Dataset):
```

```
    def __init__(self, dados, labels):
```

```
        self.dados = dados
```

```
        self.labels = labels
```

```
    def __len__(self):
```

```
        return len(self.dados)
```

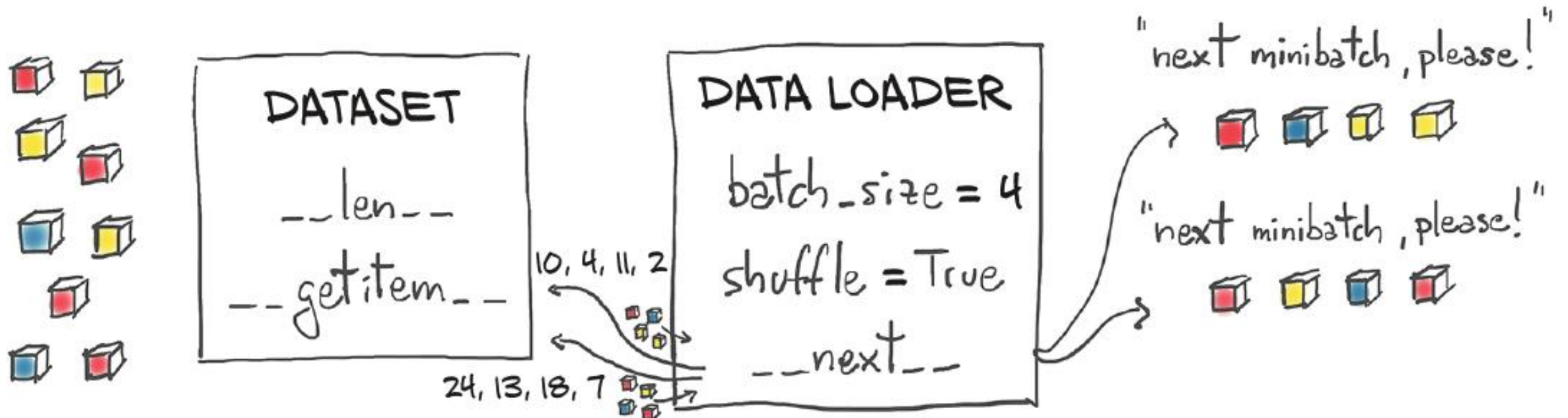
```
    def __getitem__(self, idx):
```

```
        return self.dados[idx], self.labels[idx]
```


O que é o DataLoader?

- O DataLoader é a maneira de carregar os dados em mini-lotes (mini-batches), fundamental para otimizar o treinamento de modelos.
- Principais funcionalidades:
 - Carregamento em lotes (batch_size).
 - Embaralhamento dos dados (shuffle=True).
 - Carregamento paralelo com múltiplos workers (num_workers).
 - Iteração eficiente durante o treinamento.

O que é o DataLoader?



O que é o DataLoader?

```
from torch.utils.data import DataLoader
```

```
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
```

Transformações com torchvision.transforms

- As transformações são usadas para pré-processar os dados.
- *Data Augmentation* (aumentar diversidade dos dados)
- Algumas transformações comuns:
 - ToTensor(): Converte imagens de NumPy ou PIL para Tensors.
 - Normalize(): Normaliza os valores de pixel.
 - Resize(): Redimensiona imagens para um tamanho fixo.

Transformações com torchvision.transforms

```
from torchvision import transforms
```

```
transform = transforms.Compose([  
    transforms.Resize(256),  
    transforms.RandomCrop(224),  
    transforms.ToTensor(),  
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])  
])
```

O que é datasets.ImageFolder?

- ImageFolder é uma classe do PyTorch para carregar datasets de imagens que estão organizadas em pastas de acordo com suas classes.
- Estrutura de diretórios esperada:
 - Cada subpasta representa uma classe e contém as imagens correspondentes.

```
dataset/  
├── classe_1/  
│   ├── img1.jpg  
│   └── img2.jpg  
└── classe_2/  
    ├── img3.jpg  
    └── img4.jpg
```

Sintaxe Básica de ImageFolder

- A função principal da classe ImageFolder é facilitar a organização e o carregamento de imagens para treino e validação.

```
from torchvision import datasets, transforms

transform = transforms.Compose([transforms.Resize((128, 128)), transforms.ToTensor()])
dataset = datasets.ImageFolder(root='caminho_para_o_dataset', transform=transform)
```

- Parâmetros:
 - root: Caminho para o diretório contendo as imagens organizadas por classe.
 - transform: Transformações que serão aplicadas às imagens.

Dividindo Dataset em Treino e Validação

- Como ImageFolder carrega todas as imagens de uma vez, é comum precisar dividir o dataset em treinamento e validação.

```
from torch.utils.data import random_split

# Suponha que queremos 80% dos dados para treino e 20% para validação
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
```


O que é Fine-Tuning?

- Transfer Learning: Técnica que reutiliza modelos treinados em grandes datasets (como ImageNet) para novas tarefas, economizando tempo e recursos.
- Fine-Tuning: Ajusta um modelo pré-treinado para a tarefa específica, geralmente com poucos dados.
 - Congela as camadas iniciais (para reutilizar características gerais).
 - Ajusta (treina) as últimas camadas para a nova tarefa.

O que é Fine-Tuning?

- Economia de Tempo: Reduz o tempo de treinamento, uma vez que o modelo já "aprendeu" características úteis.
- Melhor Desempenho: Resultados superiores em tarefas com poucos dados, comparado ao treinamento do zero.
- Generalização: Reaproveita características gerais aprendidas em grandes datasets como o ImageNet.

Modelos Pré-Treinados no PyTorch

- PyTorch oferece uma ampla gama de modelos pré-treinados na biblioteca `torchvision.models`:
 - ResNet, VGG, Inception, DenseNet, etc.
 - Treinados em ImageNet, contendo mais de 14 milhões de imagens em 1000 classes.

```
# define the model
model = models.vgg11(weights=VGG11_Weights.DEFAULT)
```

Estrutura do Modelo Pré-Treinado

- O modelo pré-treinado consiste em duas partes:
 - Feature Extractor: Camadas convolucionais que extraem características genéricas de imagens.
 - Classificador: Camadas totalmente conectadas que ajustam essas características para uma tarefa específica (classificação).

Fine-Tuning: Estratégia Básica

- Congelar Camadas: Manter as camadas convolucionais fixas para preservar as características genéricas já aprendidas.
- Treinar Camadas Finais: Ajustar as últimas camadas (ou adicionar novas) para a nova tarefa.

```
for param in model.parameters():  
    param.requires_grad = False # Congelar todas as camadas  
  
# Substituir a última camada (classificador) para uma nova tarefa  
model.fc = torch.nn.Linear(in_features=2048, out_features=num_classes)
```